Database Systems Lab — Week 03

Advanced SQL: Aggregations & Analytics

Mistral Vibe

February 20, 2024

Abstract

This lab explores advanced SQL concepts including aggregate functions, `GROUP BY` and `HAVING` clauses, window functions (`ROW_NUMBER`, `RANK`, `LAG`), and Common Table Expressions (CTEs). Through 10 comprehensive queries on an e-commerce dataset, we demonstrate practical applications of these techniques for business intelligence and data analysis.

Contents

1 Introduction **2**

2 Dataset Extension **2**

3 Queries **2**

3.1 Query 1: Overall Business Summary . . . 2
3.2 Query 2: Revenue by Product Category . . . 3
3.3 Query 3: Monthly Order Volume and Revenue . . . 4
3.4 Query 4: Active Cities (HAVING Clause) . . . 4
3.5 Query 5: Device Performance with HAVING . . . 5
3.6 Query 6: Ranking Orders with ROW_NUMBER . . . 6
3.7 Query 7: Overall Revenue Ranking with RANK . . . 7
3.8 Query 8: Month-over-Month Revenue Trend with LAG . . . 7
3.9 Query 9: Customer Value Segmentation using CTEs . . . 8
3.10 Query 10: Session-to-Purchase Funnel Analysis . . . 9

4 Summary of Techniques **10**

5 Learning Outcomes **11**

1 Introduction

This lab focuses on advanced SQL concepts essential for data analysis and business intelligence:

- Aggregate functions (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`)
- `GROUP BY` and `HAVING` clauses
- Window functions (`ROW_NUMBER`, `RANK`, `DENSE_RANK`, `LAG`, `LEAD`)
- Common Table Expressions (CTEs)
- Time-series and cohort analysis

2 Dataset Extension

The lab extends the e-commerce dataset with a `user_sessions` table that tracks user behavior:

```sql
CREATE TABLE user_sessions (
    session_id SERIAL PRIMARY KEY,
    customer_id INTEGER REFERENCES customers(customer_id),
    session_date DATE NOT NULL,
    pages_viewed INTEGER NOT NULL DEFAULT 1,
    duration_mins DECIMAL(6,2) NOT NULL,
    device VARCHAR(20) NOT NULL,
    converted BOOLEAN NOT NULL DEFAULT FALSE
);
```

Listing 1: User Sessions Table Schema

This table enables analysis of user engagement and conversion funnels.

3 Queries

3.1 Query 1: Overall Business Summary

This query computes key performance indicators (KPIs) for delivered orders.

```sql
SELECT COUNT(*) AS total_orders,
       SUM(total_amount) AS total_revenue,
       ROUND(AVG(total_amount), 2) AS avg_order_value,
       MAX(total_amount) AS largest_order
FROM orders
WHERE status = 'delivered';
```

Listing 2: Overall Business Summary Query

Techniques: Aggregate functions, `WHERE` clause, `ROUND`

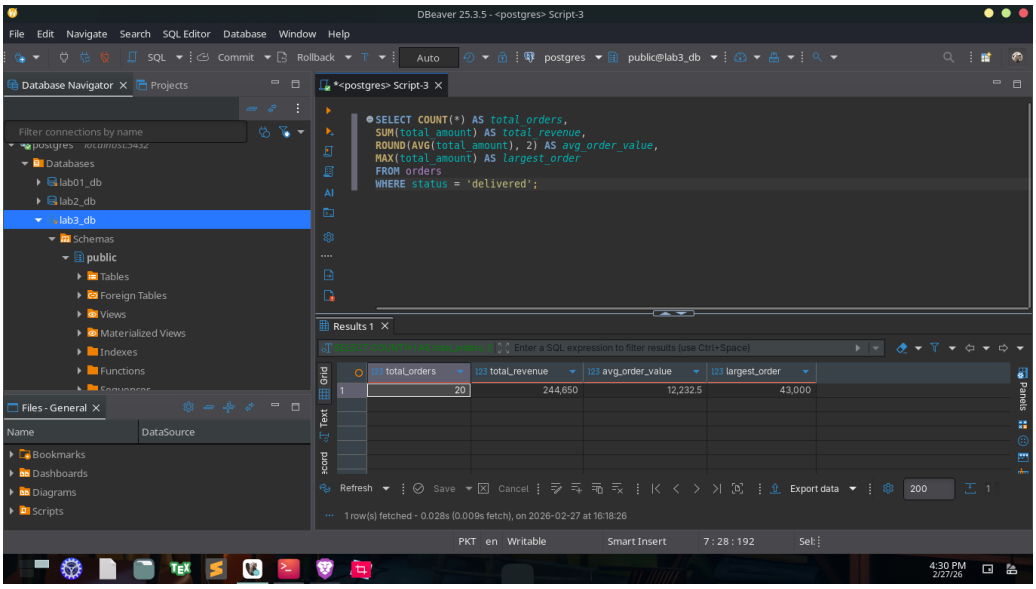

Figure 1: Overall Business Summary Results

3.2 Query 2: Revenue by Product Category

Analyzes revenue contribution across product categories using joins and aggregation.

```sql
SELECT p.category,
       COUNT(DISTINCT oi.order_id) AS orders_containing,
       SUM(oi.quantity * oi.unit_price) AS category_revenue
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.category
ORDER BY category_revenue DESC;
```

Listing 3: Revenue by Product Category Query

Techniques: `JOIN`, `GROUP BY`, `DISTINCT`, aggregation

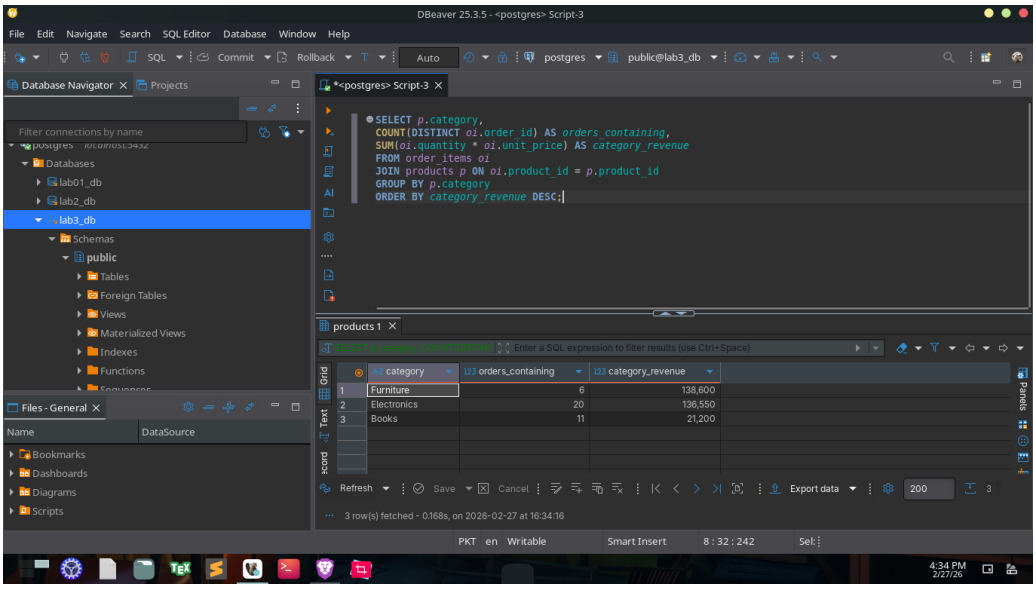

Figure 2: Revenue by Product Category

3.3 Query 3: Monthly Order Volume and Revenue

Time-series aggregation to track business performance over time.

```sql
SELECT TO_CHAR(order_date, 'YYYY-MM') AS month,
       COUNT(*) AS num_orders,
       SUM(total_amount) AS monthly_revenue
FROM orders
GROUP BY TO_CHAR(order_date, 'YYYY-MM')
ORDER BY month;
```

Listing 4: Monthly Order Volume and Revenue Query

Techniques: Date formatting (`TO_CHAR`), time-series grouping

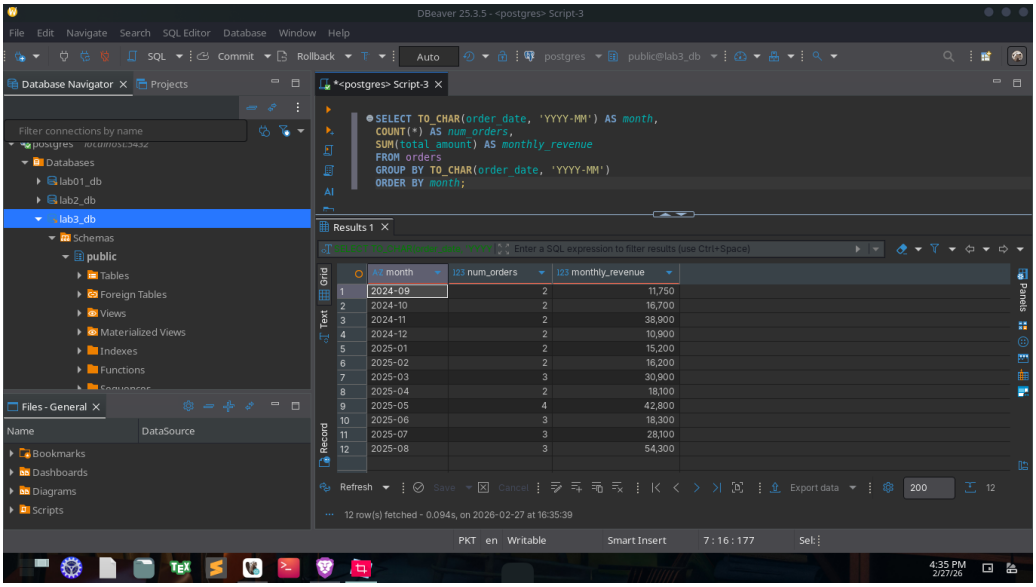

Figure 3: Monthly Order Volume and Revenue Trend

3.4 Query 4: Active Cities (HAVING Clause)

Identifies cities with more than 2 orders using the `HAVING` clause.

```sql
SELECT c.city,
       COUNT(DISTINCT c.customer_id) AS customers,
       COUNT(o.order_id) AS total_orders
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.city
HAVING COUNT(o.order_id) > 2
ORDER BY total_orders DESC;
```

Listing 5: Active Cities Query

Techniques: `HAVING` vs `WHERE`, multi-column aggregation

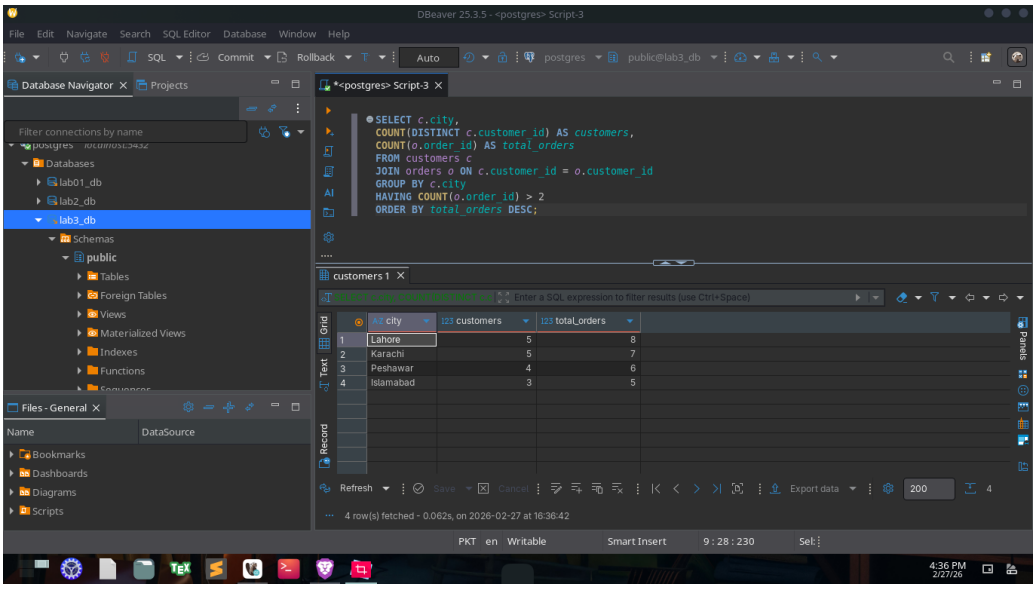

Figure 4: Active Cities with More Than 2 Orders

3.5 Query 5: Device Performance with HAVING

Analyzes user session metrics by device type with conditional aggregation.

```sql
SELECT device,
       COUNT(*) AS total_sessions,
       ROUND(AVG(duration_mins), 2) AS avg_duration,
       SUM(CASE WHEN converted THEN 1 ELSE 0 END) AS conversions,
       ROUND(100.0 * SUM(CASE WHEN converted THEN 1 ELSE 0 END) / COUNT(*), 1)
           AS conversion_rate_pct
FROM user_sessions
GROUP BY device
HAVING COUNT(*) >= 5
ORDER BY avg_duration DESC;
```

Listing 6: Device Performance Query

Techniques: Conditional aggregation (`CASE`), percentage calculations

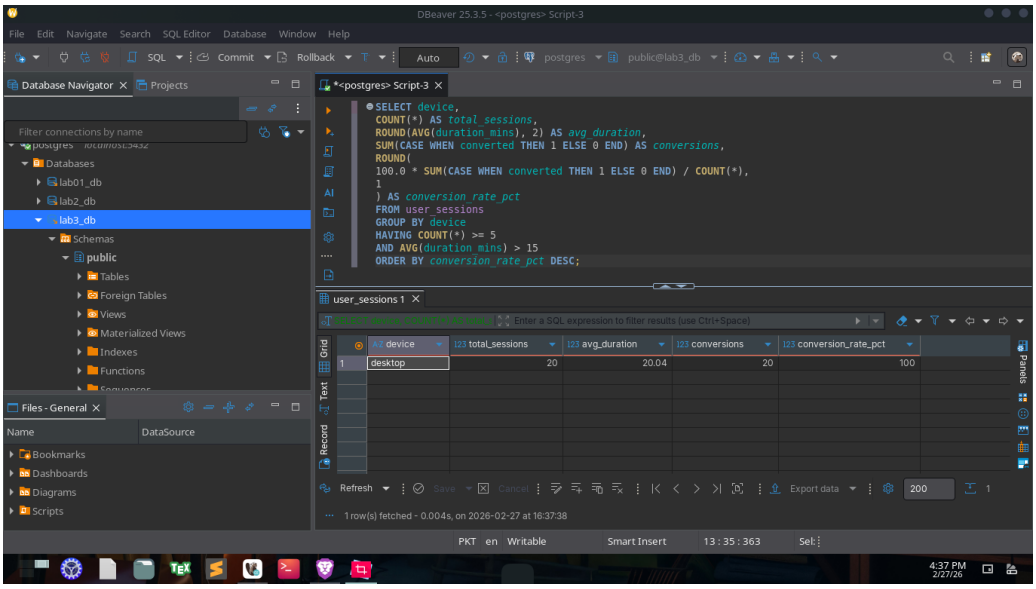

Figure 5: Device Performance Metrics

3.6 Query 6: Ranking Orders with ROW_NUMBER

Demonstrates window functions to rank orders within each customer.

```sql
SELECT customer_id,
       order_id,
       order_date,
       total_amount,
       ROW_NUMBER() OVER (
           PARTITION BY customer_id
           ORDER BY total_amount DESC
       ) AS rank_within_customer
FROM orders
ORDER BY customer_id, rank_within_customer;
```

Listing 7: Ranking Orders with ROW$_NUMBER$

Techniques: Window functions, `ROW_NUMBER()`, `PARTITION BY`

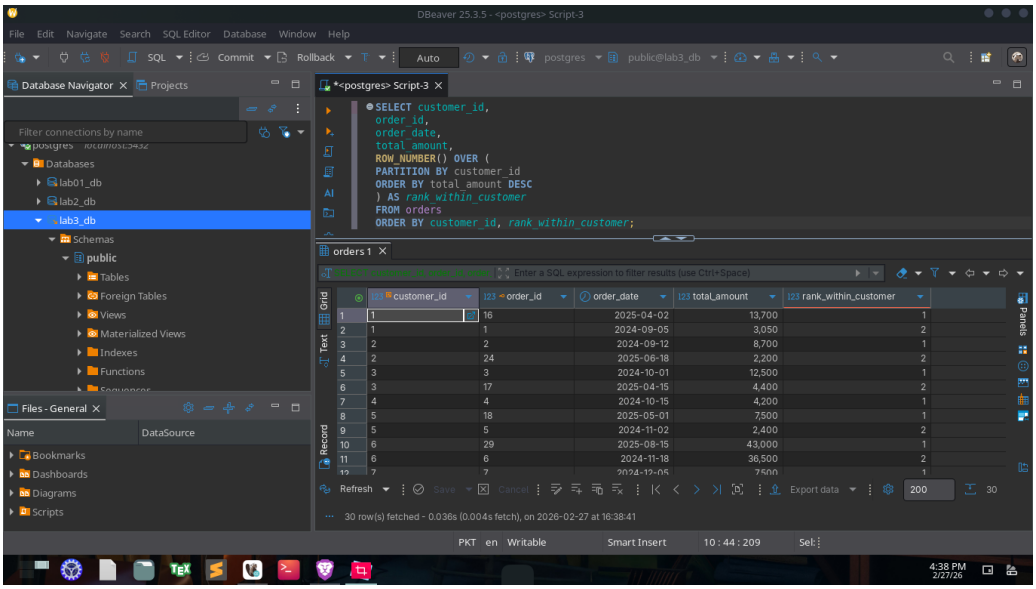

Figure 6: Customer Orders Ranked by Value

Query 6b: Each Customer's Highest-Value Order

Using CTE with `ROW_NUMBER` to find top orders.

```sql
WITH ranked_orders AS (
    SELECT customer_id,
           order_id,
           order_date,
           total_amount,
           ROW_NUMBER() OVER (
               PARTITION BY customer_id
               ORDER BY total_amount DESC
           ) AS rank_within_customer
    FROM orders
)
SELECT customer_id, order_id, order_date, total_amount
FROM ranked_orders
WHERE rank_within_customer = 1
ORDER BY customer_id;
```

Listing 8: Highest-Value Order per Customer

3.7 Query 7: Overall Revenue Ranking with RANK

Compares `RANK` vs `DENSE_RANK` for order revenue ranking.

```sql
SELECT order_id,
       customer_id,
       total_amount,
       RANK() OVER (ORDER BY total_amount DESC) AS rank,
       DENSE_RANK() OVER (ORDER BY total_amount DESC) AS dense_rank
FROM orders
ORDER BY total_amount DESC
LIMIT 15;
```

Listing 9: Revenue Ranking with RANK and DENSE$_RANK$

Techniques: `RANK()`, `DENSE_RANK()`, window ordering

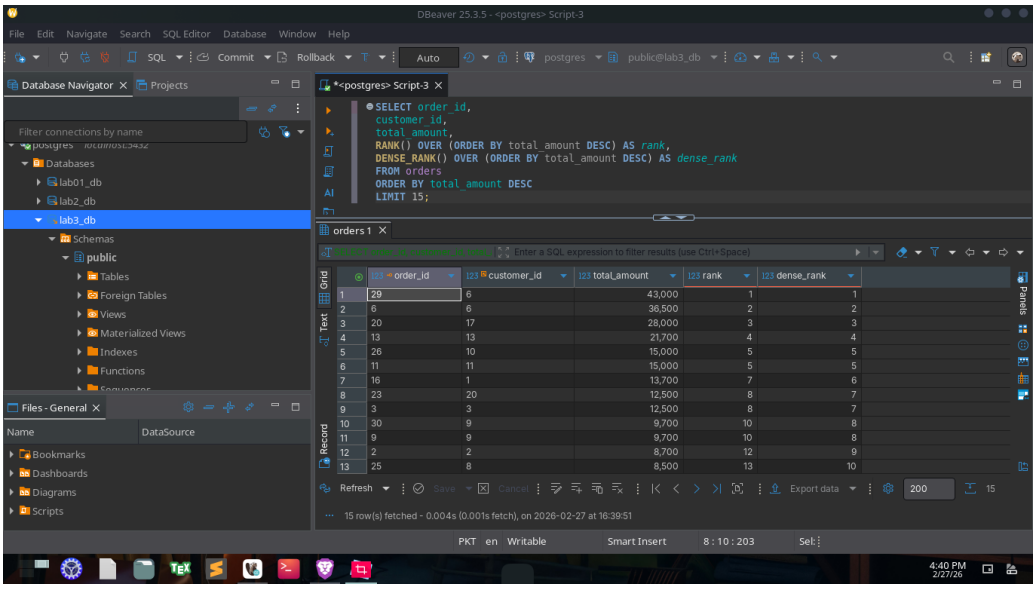

Figure 7: Order Revenue Ranking Comparison

3.8 Query 8: Month-over-Month Revenue Trend with LAG

Uses the `LAG` window function to compute period-over-period changes.

```sql
WITH monthly_revenue AS (
    SELECT TO_CHAR(order_date, 'YYYY-MM') AS month,
           SUM(total_amount) AS revenue
    FROM orders
    WHERE status = 'delivered'
    GROUP BY TO_CHAR(order_date, 'YYYY-MM')
)
SELECT month,
       revenue,
       LAG(revenue) OVER (ORDER BY month) AS prev_month,
       revenue - LAG(revenue) OVER (ORDER BY month) AS absolute_change,
       ROUND(
           100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
           / NULLIF(LAG(revenue) OVER (ORDER BY month), 0),
           1
       ) AS pct_change
FROM monthly_revenue
ORDER BY month;
```

Listing 10: Month-over-Month Trend with LAG

Techniques: CTEs, `LAG()`, null-safe division (`NULLIF`)

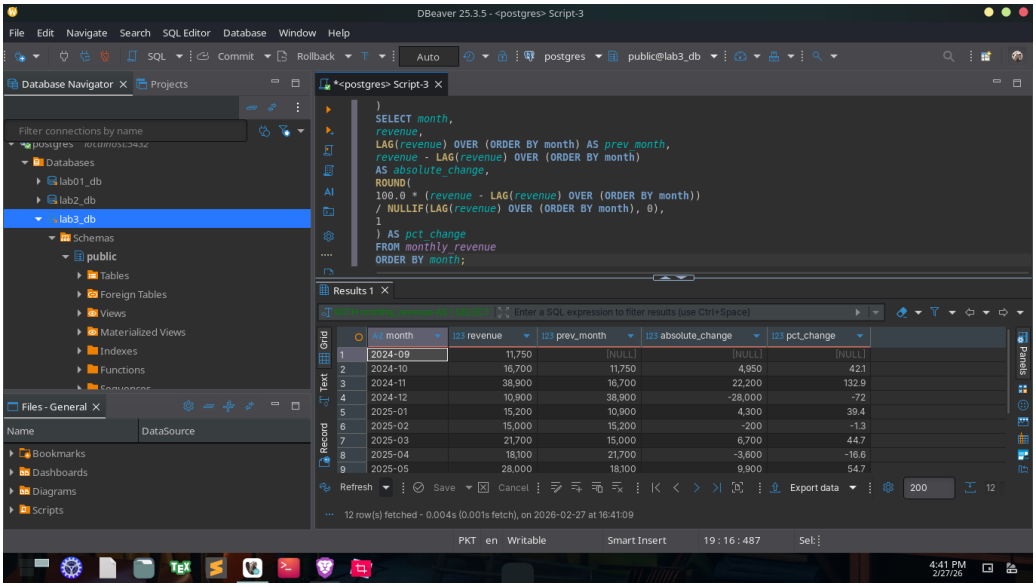

Figure 8: Month-over-Month Revenue Trend Analysis

3.9 Query 9: Customer Value Segmentation using CTEs

Multi-level CTE for customer tier classification and revenue analysis.

```sql
WITH customer_spend AS (
    SELECT c.customer_id,
           c.name,
           c.city,
           COALESCE(SUM(o.total_amount), 0) AS total_spent
    FROM customers c
    LEFT JOIN orders o ON c.customer_id = o.customer_id
    AND o.status = 'delivered'
    GROUP BY c.customer_id, c.name, c.city
),
customer_tiers AS (
    SELECT *,
           CASE
               WHEN total_spent > 30000 THEN 'VIP'
               WHEN total_spent > 10000 THEN 'High Value'
               WHEN total_spent > 0 THEN 'Active'
               ELSE 'Never Purchased'
           END AS tier
    FROM customer_spend
)
SELECT tier,
       COUNT(*) AS customers,
       SUM(total_spent) AS total_revenue,
       ROUND(100.0 * SUM(total_spent) / SUM(SUM(total_spent)) OVER (), 1)
           AS pct_of_revenue
FROM customer_tiers
GROUP BY tier
ORDER BY total_revenue DESC;
```

Listing 11: Customer Value Segmentation

Techniques: Multi-level CTEs, `COALESCE`, window aggregates

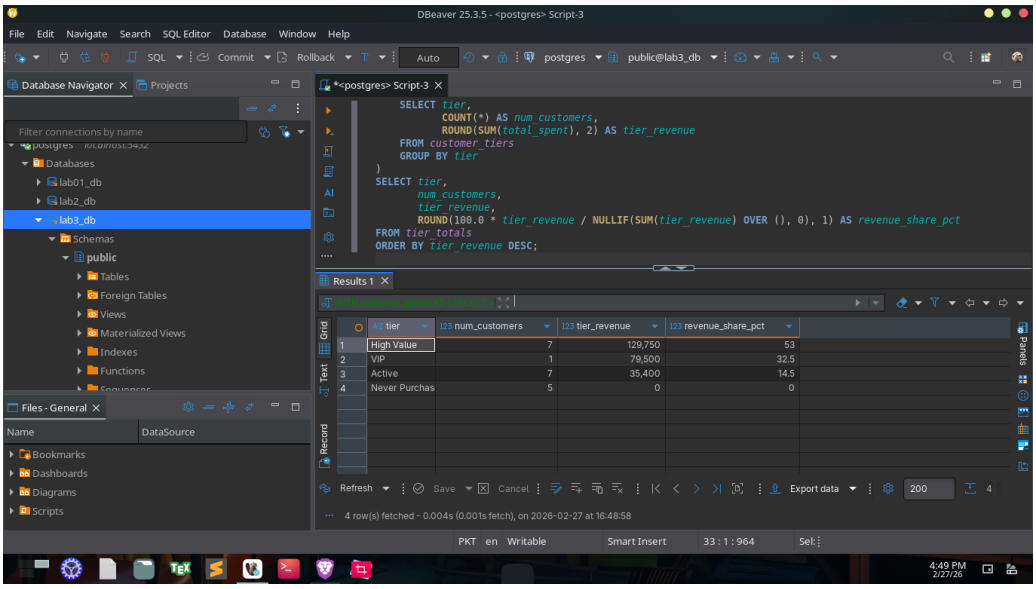

Figure 9: Customer Value Segmentation by Tier

3.10 Query 10: Session-to-Purchase Funnel Analysis

Complex CTE combining session and order data for funnel metrics.

```sql
WITH session_summary AS (
    SELECT customer_id,
           COUNT(*) AS total_sessions,
           SUM(pages_viewed) AS total_pages,
           ROUND(AVG(duration_mins), 2) AS avg_duration,
           SUM(CASE WHEN converted THEN 1 ELSE 0 END) AS converted_sessions
    FROM user_sessions
    GROUP BY customer_id
),
order_summary AS (
    SELECT customer_id,
           COUNT(*) AS total_orders,
           SUM(total_amount) AS total_spent
    FROM orders
    WHERE status = 'delivered'
    GROUP BY customer_id
),
combined AS (
    SELECT c.name,
           c.city,
           s.total_sessions,
           s.total_pages,
           s.avg_duration,
           s.converted_sessions,
           COALESCE(o.total_orders, 0) AS total_orders,
           COALESCE(o.total_spent, 0) AS total_spent
    FROM session_summary s
    JOIN customers c ON s.customer_id = c.customer_id
    LEFT JOIN order_summary o ON s.customer_id = o.customer_id
)
SELECT name,
       city,
       total_sessions,
       total_pages,
       avg_duration,
       total_orders,
```

```sql
        total_spent,
        ROUND(
            CASE WHEN total_sessions > 0
                 THEN 100.0 * total_orders / total_sessions
                 ELSE 0
            END, 1
        ) AS orders_per_100_sessions
FROM combined
ORDER BY total_sessions DESC, orders_per_100_sessions ASC;
```

Listing 12: Session-to-Purchase Funnel Analysis

Techniques: Multiple CTEs, funnel analysis, conversion metrics

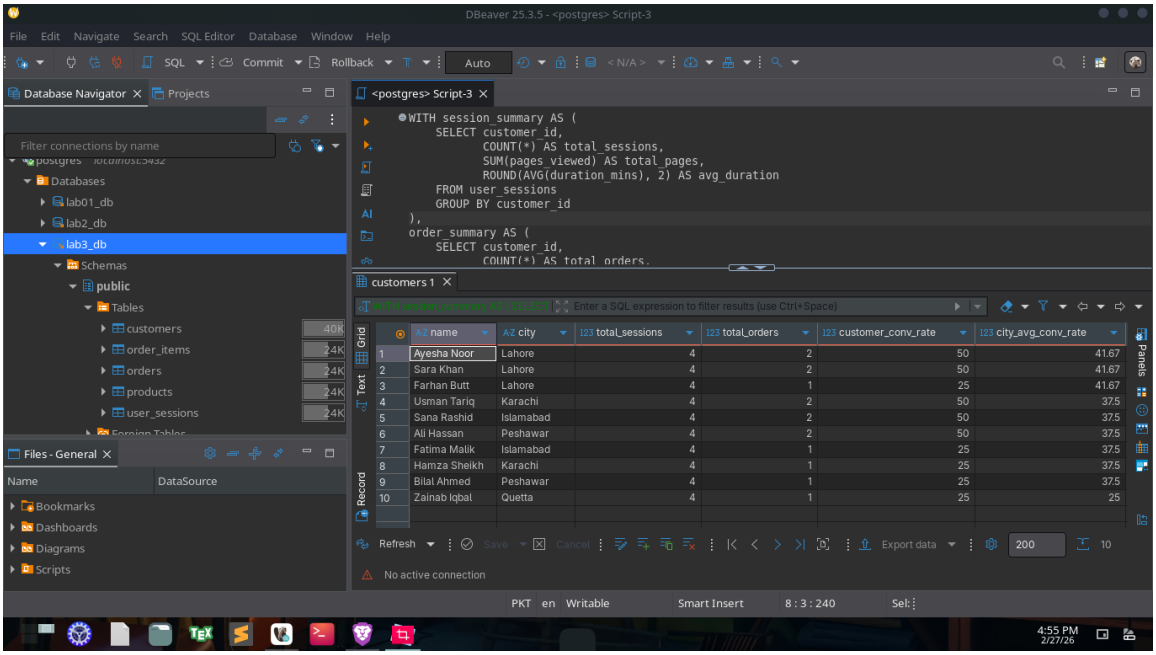

Figure 10: Session-to-Purchase Funnel Analysis

4 Summary of Techniques

Table 1: SQL Techniques Demonstrated in Each Query

Query	Primary Technique	Key Functions
1	Aggregate KPIs	COUNT, SUM, AVG, MAX
2	Join & Group By	JOIN, GROUP BY, DISTINCT
3	Time-series	TO_CHAR, GROUP BY
4	HAVING Clause	HAVING, COUNT
5	Conditional Aggregation	CASE, HAVING
6	Window Ranking	ROW_NUMBER, PARTITION BY
7	Rank Comparison	RANK, DENSE_RANK
8	Trend Analysis	LAG, CTE
9	Customer Segmentation	Multi-level CTE, CASE
10	Funnel Analysis	Multiple CTEs, JOINs

5 Learning Outcomes

By completing this lab, you should now be able to:

- Understand the difference between `WHERE` and `HAVING` clauses
- Apply window functions for analytical queries
- Write readable complex queries using CTEs
- Perform time-series and trend analysis
- Build customer segmentation and funnel analysis queries
- Analyze query performance with `EXPLAIN ANALYZE`

Appendix: Files

- `lab3_extension.sql` — Dataset extension script
- `queries.sql` — All 10 lab queries
- `performance_analysis.md` — Query performance documentation